

软件过程改进方案设计文档

学生：阙嘉鑫 学号：2023141461200 课程：软件过程与管理 四川大学软件学院

一、改进背景与目标

1.1 改进背景

本项目为第三方游戏饰品交易网站，支持 CS2/Steam 等主流游戏饰品的上架、求购、交易撮合、订单管理、资金结算及风控审核功能。原有软件过程基于 ISO/IEC 12207 裁剪，包含 7 个阶段：项目启动与规划、需求分析与规格定义、系统设计、实现与编码、集成与测试、部署与交付、维护与收尾。

根据《AI 技术应用场景匹配报告》的分析结果，本方案针对编码实现、测试验证、需求分析三个高优先级场景进行过程改进设计。

1.2 改进目标

- 编码效率提升 50%以上，代码缺陷率降低 30%
- 测试用例编写时间减少 60%，测试覆盖率达到 90%以上
- 需求文档编写时间减少 50%，需求缺陷发现率提升 40%
- 整体项目交付周期缩短 30%-40%

二、新增 AI 相关角色定义

在原有项目团队基础上，新增以下 AI 相关角色：

角色名称	职责描述	技能要求
AI 工具协调员	负责 AI 工具的选型、配置与团队培训，确保 AI 工具与开发流程的有效集成	熟悉主流 AI 开发工具，具备工具评估与集成能力
AI 输出审核员	对 AI 生成的代码、测试用例、需求	具备扎实的软件工程基础，了解 AI

	文档进行质量审核，确保输出符合项目标准	输出的常见问题与局限性
AI 训练师（兼任）	根据项目特点对 AI 工具进行 Prompt 优化和上下文调优，提升 AI 输出质量	掌握 Prompt Engineering 技巧，了解项目业务领域

注：考虑到课程项目团队规模较小，上述角色由团队成员兼任，每人承担 1-2 个 AI 相关角色。

三、过程重构方案

3.1 需求分析与规格定义阶段 — 改进方案

改进前流程：

需求调研 → 用例建模 → 需求评审 → 合规确认

改进后流程：

需求调研 → **AI 辅助需求提取** → **AI 生成需求文档初稿** → 人工修订 → **AI 一致性验证** → 需求评审 → 合规确认

具体改进措施：

- 1. **AI 辅助需求提取：**使用 LLM 对用户访谈记录、竞品分析文档进行自然语言处理，自动提取功能需求和非功能需求，形成结构化需求列表。
- 2. **AI 生成需求文档初稿：**基于提取的结构化需求，使用通义千问/GPT-4 自动生成符合模板的需求规格说明书初稿。
- 3. **AI 一致性验证：**利用 AI 对需求文档进行语义分析，检测需求间的矛盾、遗漏和歧义，生成一致性检查报告。

AI 输出物质量标准：

- AI 生成的需求文档需经人工审核确认，准确率需达到 95%以上方可进入评审环节
- AI 一致性检查报告需覆盖所有功能需求，遗漏率不超过 5%

3.2 实现与编码阶段 — 改进方案

改进前流程：

模块开发 → API 对接 → 单元测试 → 代码管控

改进后流程：

AI 辅助模块开发 → API 对接 → **AI 生成单元测试** → **AI 代码审查** → 人工复核 → 代码管控

具体改进措施：

4. **AI 辅助模块开发：**开发者使用 GitHub Copilot/Cursor 进行代码编写，AI 根据上下文自动补全代码、生成函数实现。开发者负责架构把控和业务逻辑审核。
5. **AI 生成单元测试：**使用 GitHub Copilot 或 Diffblue Cover 为每个模块自动生成单元测试用例，覆盖正常路径和异常路径。
6. **AI 代码审查：**集成 CodeRabbit 进行自动化代码审查，检测代码缺陷、安全漏洞、性能问题和编码规范违规。

AI 输出物质量标准：

- AI 生成的单元测试覆盖率需达到 80%以上
- AI 代码审查发现的问题需经人工确认后修复，误报率控制在 20%以内
- 所有 AI 生成的代码必须经过人工复核后方可合入主干

3.3 集成与测试阶段 — 改进方案

改进前流程：

模块联调 → 功能测试 → 性能测试 → 风控专项测试

改进后流程：

模块联调 → **AI 生成测试用例** → **AI 执行自动化测试** → 功能测试 → 性能测试 → 风控专项测试
→ **AI 缺陷分析**

具体改进措施：

7. **AI 生成测试用例：**基于需求文档和代码结构，使用 WHartTest/LLM 自动生成测试用例，包括功能测试用例、边界测试用例和异常测试用例。
8. **AI 执行自动化测试：**使用 Testin XAgent 执行 UI 自动化测试，支持测试脚本的自愈修复。
9. **AI 缺陷分析：**对测试中发现的缺陷，使用 AI 进行根因分析和影响范围评估，自动生成缺陷修复建议。

AI 输出物质量标准:

- AI 生成的测试用例需覆盖所有功能需求点，需求覆盖率不低于 90%
- AI 自动化测试通过率需作为质量门禁指标，低于 95%不得进入部署阶段

四、工具链选型

改进环节	工具名称	核心功能	集成方式	预期效率提升
需求文档生成	通义千问/GPT-4	自然语言理解与文档生成	Web API 调用，手动交互	需求文档编写时间减少 50%
需求一致性验证	GitHub Copilot Chat	语义分析、矛盾检测	IDE 内嵌插件	需求缺陷发现率提升 40%
代码生成与补全	GitHub Copilot	上下文代码补全与生成	VS Code/JetBrains 插件	编码效率提升 55%
代码审查	CodeRabbit	自动代码审查、缺陷检测	GitHub/GitLab PR 集成	代码审查效率提升 3 倍
单元测试生成	GitHub Copilot / Diffblue	自动生成单元测试	IDE 插件 / CI 集成	测试用例编写时间减少 60%
自动化测试	Testin XAgent	UI 自动化测试、脚本自愈	SaaS 平台集成	测试维护成本降低 50%
缺陷分析	通义千问	缺陷根因分析、修复建议	Web API 调用	缺陷定位时间缩短 40%

五、改进后过程流程图

[流程图 - 详见源 Markdown 文档]

六、风险评估与应对策略

风险类别	风险描述	影响程度	应对策略
数据安全	AI 工具可能将项目代码、业务数据传输至外部服务器	高	使用本地部署的 AI 工具或选择有数据保护承诺的服务；敏感数据脱敏后再输入 AI
模型偏见	AI 生成的代码或测试用例可能存在偏见，覆盖场景不全面	中	建立 AI 输出人工审核机制，确保关键逻辑由人工验证
技术依赖	过度依赖 AI 工具可能导致	中	保持核心模块的人工编写

	团队成员基础能力下降		能力，AI 仅作为辅助工具
输出质量不稳定	AI 输出质量受 Prompt 质量、上下文长度等因素影响	中	建立 Prompt 模板库，积累项目特定的最佳实践
工具兼容性	AI 工具与现有技术栈可能存在兼容性问题	低	提前进行工具兼容性测试，准备替代方案

七、预期效果总结

通过上述改进方案的实施，预期可实现以下效果：

- 10. **效率提升**：整体开发周期缩短 30%-40%，其中编码阶段效率提升 55%，测试阶段效率提升 60%
- 11. **质量提升**：代码缺陷率降低 30%，测试覆盖率提升至 90%以上，需求缺陷发现率提升 40%
- 12. **成本优化**：减少重复性劳动，使团队成员能够将精力集中在架构设计、业务逻辑等核心工作上
- 13. **能力沉淀**：形成 AI 辅助开发的最佳实践和 Prompt 模板库，为后续项目提供可复用的经验